

Machine Learning e Pattern Recognition – Exam Collection

Luca & Mellow

7 maggio 2026

Indice

1	PCA	2
2	LDA	3
3	PCA vs LDA	5
4	Generative Gaussian Moldes	8

1 PCA

Motivation. Dimensionality reduction maps data from an n -dimensional space to an m -dimensional one, with $m \ll n$. The practical purposes are four-fold: **to compress information, remove unwanted variability (noise), simplify classification by reducing the number of parameters to estimate, and visualize the data.**

PCA is the standard linear and unsupervised technique for this task. The matrix $P \in \mathbb{R}^{n \times m}$ with orthonormal columns minimizes the average reconstruction error:

$$P^* = \arg \min_P \frac{1}{K} \sum_i \|\mathbf{x}_i - PP^T \mathbf{x}_i\|^2 \quad \text{s.t. } P^T P = I$$

Equivalently, it maximizes the projected variance:

$$\tilde{L}(P) = \text{Tr} \left(P^T \underbrace{\frac{1}{K} \sum_i \mathbf{x}_i \mathbf{x}_i^T}_C P \right)$$

The solution consists of the m eigenvectors of C with the largest eigenvalues. From the decomposition $C = U \Sigma U^T$:

$$P^* = [\mathbf{u}_1, \dots, \mathbf{u}_m]$$

The result is a coordinate system where each axis captures the maximum remaining variance, and the variance along each axis corresponds to the associated eigenvalue.

Non-centered data. If the data are not zero-mean, the first eigenvector points towards the dataset mean — not informative. The mean $\bar{\mathbf{x}}$ should always be subtracted, and C becomes the empirical covariance matrix:

$$C = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$$

Choice of m . m is a hyperparameter: it does not emerge from the PCA solution itself. Two approaches: cross-validation, or a criterion on explained variance — choose the smallest m such that:

$$\frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} \geq t \quad (\text{e.g. } t = 0.95)$$

The choice of m introduces a trade-off. If m is too large, too many dimensions remain, increasing the number of parameters to estimate in the downstream classifier and the risk of overfitting. If m is too small, discriminant directions may be lost, causing underfitting. Cross-validation allows balancing this trade-off for the specific task.

Limitations. Computing C is expensive in high-dimensional spaces. Most importantly: being unsupervised, PCA does not guarantee to preserve discriminant directions — directions of highest variance do not necessarily coincide with those useful for class separation. **A further practical advantage is that PCA automatically eliminates directions with zero or very low variance, which could cause numerical instability in some classifiers** (e.g., division by zero variance).

2 LDA

Motivation. LDA is a linear **supervised** technique for dimensionality reduction and classification. **Unlike PCA, which maximizes variance without knowledge of class labels, LDA exploits class information to find directions that explicitly separate groups.** The goal is to find the direction $\mathbf{w}$ that maximizes the separation between classes relative to the within-class scatter:

$$\max_{\mathbf{w}} L(\mathbf{w}) = \frac{s_B}{s_W} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}}$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^K n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

S_B measures how far the class means are from the global mean; S_W measures how spread out the samples are around their class mean. Maximizing the ratio means finding a projection where classes are compact internally and far apart from each other.

Solution derivation. Setting $\nabla_{\mathbf{w}} L = 0$, dividing by the scalar $(\mathbf{w}^T S_W \mathbf{w})$ and multiplying by S_W^{-1} , leads to the eigenvalue problem:

$$S_W^{-1} S_B \mathbf{w} = \lambda(\mathbf{w}) \mathbf{w}, \quad \lambda(\mathbf{w}) = L(\mathbf{w})$$

Maximizing L is thus equivalent to maximizing λ : the optimal direction is the eigenvector of $S_W^{-1} S_B$ with the largest eigenvalue.

Binary case. With two classes, $S_B = k(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T$, which has rank 1, and the solution simplifies in closed form:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

This formula has a direct interpretation: compute the difference between the two class means and "correct" it for within-class scatter via S_W^{-1} . If the classes have the same within-class scatter ($S_W = I$), the optimal direction is simply the one joining the two means.

The classification rule assigns $\mathbf{x}_t$ to the closest class in the projected space, with threshold $t = \frac{m_1 + m_2}{2}$:

$$C(\mathbf{x}_t) = \begin{cases} C_1 & \mathbf{w}^T \mathbf{x}_t < t \\ C_2 & \mathbf{w}^T \mathbf{x}_t \geq t \end{cases}$$

The threshold t is the mean of the projected class means: it is the optimal choice under the assumption of equal within-class variance and equal prior. If the priors differ, the threshold must be shifted accordingly.

Extension to m dimensions. The m most discriminant directions are collected in $W \in \mathbb{R}^{n \times m}$. The criterion becomes:

$$\max_W \text{Tr}(\tilde{S}_W^{-1} \tilde{S}_B), \quad \tilde{S}_B = W^T S_B W, \quad \tilde{S}_W = W^T S_W W$$

The solution consists of the m eigenvectors of $S_W^{-1}S_B$ with the largest eigenvalues. **An important structural constraint: the rank of S_B is at most $K - 1$, because it is defined by K means constrained to sum to the global mean. As a result, the maximum number of discriminant directions that can be extracted is $K - 1$, regardless of the original data dimension. For binary classification, there is always only one direction.**

Limitations and practical considerations. LDA assumes that the within-class distributions are Gaussian with the same covariance matrix — an assumption often violated in practice. S_W can become singular when the number of samples is less than the data dimension, making S_W^{-1} not computable. **A common solution is to apply PCA as a pre-processing step before LDA: PCA reduces dimensionality by eliminating directions with zero or near-zero variance, making S_W invertible and reducing the risk of overfitting. However, care must be taken not to eliminate discriminant directions with PCA before LDA can find them — the number of PCA components should be chosen carefully, typically via cross-validation.**

3 PCA vs LDA

Both **PCA** and **LDA** are linear dimensionality reduction techniques that map data from an n -dimensional feature space to a lower m -dimensional space ($m \ll n$), but they differ fundamentally in supervision, objective, and the nature of the directions they find.

Goals and Formulation

PCA is an *unsupervised* technique. Its goal is to find an orthogonal projection matrix $P \in \mathbb{R}^{n \times m}$ that best preserves the structure of the data, without any knowledge of class labels. Data is first centered by subtracting the mean $\bar{\mathbf{x}}$, and then PCA seeks directions that capture the most variance. The projection of a sample is $\mathbf{y} = P^T \mathbf{x}$, and the reconstruction in the original space is $\hat{\mathbf{x}} = P\mathbf{y}$.

LDA is a *supervised* technique. Its goal is to find a projection matrix $W \in \mathbb{R}^{n \times m}$ such that the projected data has maximum separation between classes

and minimum spread within each class. It explicitly uses class labels to compute class means $\boldsymbol{\mu}_c$ and the global mean $\boldsymbol{\mu}$. The projection of a sample is $\tilde{\mathbf{x}} = W^T \mathbf{x}$.

Training Objective

The PCA objective is to **minimize the average reconstruction error**:

$$P^* = \arg \min_P \frac{1}{K} \sum_{i=1}^K \|\mathbf{x}_i - PP^T \mathbf{x}_i\|^2, \quad \text{s.t. } P^T P = I$$

This is equivalent to **maximizing the projected variance**:

$$\tilde{L}(P) = \text{Tr}(P^T C P)$$

where $C = \frac{1}{K} \sum_i (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^T$ is the empirical covariance matrix. The constraint $P^T P = I$ ensures orthonormality of the components.

The LDA objective is to **maximize the Fisher ratio** — the ratio of between-class variability to within-class variability. **For the multi-direction case, the criterion is:**

$$L = \text{Tr}(\tilde{S}_W^{-1} \tilde{S}_B) = \text{Tr}((W^T S_W W)^{-1} (W^T S_B W))$$

where the scatter matrices are:

$$S_B = \frac{1}{N} \sum_{c=1}^K n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T, \quad S_W = \frac{1}{N} \sum_{c=1}^K \sum_{i=1}^{n_c} (\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)(\mathbf{x}_{c,i} - \boldsymbol{\mu}_c)^T$$

Unlike PCA, LDA does *not* require W to be orthogonal.

Characteristics of the Directions

The table below summarizes the key properties of the directions found by each method:

Property	PCA Principal Components	LDA Discriminant Directions
Computed from	Eigenvectors of covariance matrix C	Eigenvectors of $S_W^{-1}S_B$
Ordering	By descending eigenvalue (= variance)	By descending eigenvalue (= Fisher ratio)
Orthogonality	Guaranteed ($P^T P = I$)	Not required
Max number of directions	Up to n (feature dimensionality)	At most $K - 1$ (number of classes minus one)
Label-awareness	No — may not be discriminant	Yes — explicitly maximizes class separation
Interpretability	Directions of highest variance	Directions of highest class separability

The $K - 1$ limit on LDA directions follows directly from the rank of S_B : since S_B is a sum of K rank-one matrices centered around the global mean, it has at most $K - 1$ non-zero eigenvalues. In contrast, PCA can extract up to n components, selected by retaining enough to explain a target fraction t of total variance:

$$\min_m m \quad \text{s.t.} \quad \frac{\sum_{i=1}^m \sigma_i}{\sum_{i=1}^n \sigma_i} \geq t$$

Use in Classification

PCA for classification is used as a *preprocessing step*. It reduces dimensionality before feeding data into a separate classifier (e.g., logistic regression, k-NN). Because PCA is unsupervised, there is no guarantee the retained components are discriminant — variance-maximizing directions and class-separating directions can differ significantly.

LDA for classification works directly. For a binary problem, the optimal direction is:

$$\mathbf{w} \propto S_W^{-1}(\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)$$

and the classification rule assigns sample $\mathbf{x}_t$ to class C_1 if $\mathbf{w}^T \mathbf{x}_t < t$, where the threshold is the midpoint of the projected class means:

$$t = \frac{m_1 + m_2}{2}, \quad m_c = \mathbf{w}^T \boldsymbol{\mu}_c$$

This is equivalent to a nearest-centroid classifier in the projected space. LDA implicitly assumes that within-class distributions are Gaussian with equal covariance, so the linear decision boundary is optimal under those assumptions.

A common practical strategy is to **apply PCA first, then LDA**: PCA removes noise and reduces the risk of S_W becoming singular (which happens

when the number of features is large relative to the number of samples), and LDA then finds the most discriminant subspace within the PCA-reduced space.

4 Generative Gaussian Moldes

Motivation. Generative Gaussian Models (GGM) address closed-set classification by explicitly modeling the data-generation process for each class. **Unlike discriminative models that learn decision boundaries directly, generative models assume a probabilistic class-conditional density and apply Bayes' theorem to compute posterior probabilities.** The fundamental assumption is that the likelihood of the data given a class follows a multivariate Gaussian distribution, allowing us to exploit the well-understood properties of Gaussians to derive tractable classification rules.

Probabilistic framework. Classification is performed by computing the posterior probability:

$$P(C_t = c | X_t = \mathbf{x}_t) = \frac{f_{X|C}(\mathbf{x}_t | c) \cdot P(C = c)}{\sum_{c'} f_{X|C}(\mathbf{x}_t | c') \cdot P(C = c')}$$

The classifier assigns $\mathbf{x}_t$ to the class with the highest posterior. The joint density factorizes as:

$$f_{X,C}(\mathbf{x}, c) = f_{X|C}(\mathbf{x} | c) \cdot P(C = c)$$

The class prior $P(C = c)$ is determined by the application; the class likelihood $f_{X|C}(\mathbf{x} | c)$ is learned from data.

Mathematical formulation. GGM assumes that each class likelihood is a multivariate Gaussian:

$$(X_t | C_t = c) \sim \mathcal{N}(\boldsymbol{\mu}_c, \boldsymbol{\Sigma}_c)$$

The log-likelihood per sample in class c is:

$$\log f_{X|C}(\mathbf{x} | c) = -\frac{D}{2} \log 2\pi - \frac{1}{2} \log |\boldsymbol{\Sigma}_c| - \frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c)$$

Training via ML estimation. Parameters $\boldsymbol{\mu}_c$ and $\boldsymbol{\Sigma}_c$ are estimated by maximizing the log-likelihood class by class. The closed-form ML solutions are:

$$\boldsymbol{\mu}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} \mathbf{x}_i, \quad \boldsymbol{\Sigma}_c^* = \frac{1}{N_c} \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

where N_c is the number of samples in class c . **Each class is estimated independently: the sample mean is the class center, and the sample covariance captures the within-class spread.**

Inference: binary classification and LLR. For a binary decision, the classifier compares the log-likelihood ratio (LLR) to a threshold derived from the log-odds:

$$\text{llr}(\mathbf{x}_t) = \log \frac{f_{X|C}(\mathbf{x}_t|c_1)}{f_{X|C}(\mathbf{x}_t|c_0)} \geq -\log \frac{P(C=c_1)}{P(C=c_0)}$$

The LLR is a **quadratic function of $\mathbf{x}$** :

$$\text{llr}(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} + \mathbf{x}^T \mathbf{b} + c$$

where the quadratic term $A = -\frac{1}{2}(\boldsymbol{\Lambda}_1 - \boldsymbol{\Lambda}_0)$ vanishes only when the two classes share the same covariance ($\boldsymbol{\Sigma}_1 = \boldsymbol{\Sigma}_0$). This produces a **quadratic decision boundary**, characteristic of QDA (Quadratic Discriminant Analysis).

Inference: multiclass classification. For $K > 2$ classes, assign the sample to:

$$c_t^* = \arg \max_h [\log f_{X|C}(\mathbf{x}_t|h) + \log P(C=h)]$$

The classifier outputs class-conditional log-likelihoods; the prior term $\log P(C=h)$ is application-dependent and can be adjusted at deployment time.

Model variants. GGM encompasses several important variants:

- **Full covariance (QDA):** Each class has its own full covariance matrix $\boldsymbol{\Sigma}_c$. Highly flexible but requires many samples; decision boundary is quadratic.

- **Tied covariance (LDA-like):** All classes share a common covariance Σ :

$$\Sigma^* = \frac{1}{N} \sum_c \sum_{i:c_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c^*)(\mathbf{x}_i - \boldsymbol{\mu}_c^*)^T$$

This reduces the number of parameters and produces a **linear decision boundary** (the quadratic term A cancels). More stable with limited data.

- **Naive Bayes Gaussian:** Each class has a diagonal covariance, assuming feature independence. Simple and computationally efficient, but assumes uncorrelated features.

Limitations. GGM assumes all data within a class are drawn from a Gaussian distribution — a strong assumption often violated in practice. **Estimating a full covariance matrix requires $\frac{D(D+1)}{2}$ parameters**, which becomes prohibitive in high dimensions and with limited data. The covariance matrix must be invertible; if $N < D$, standard ML estimation fails. A common solution is to apply PCA beforehand to reduce dimensionality and ensure invertibility, though care must be taken not to eliminate discriminant directions.